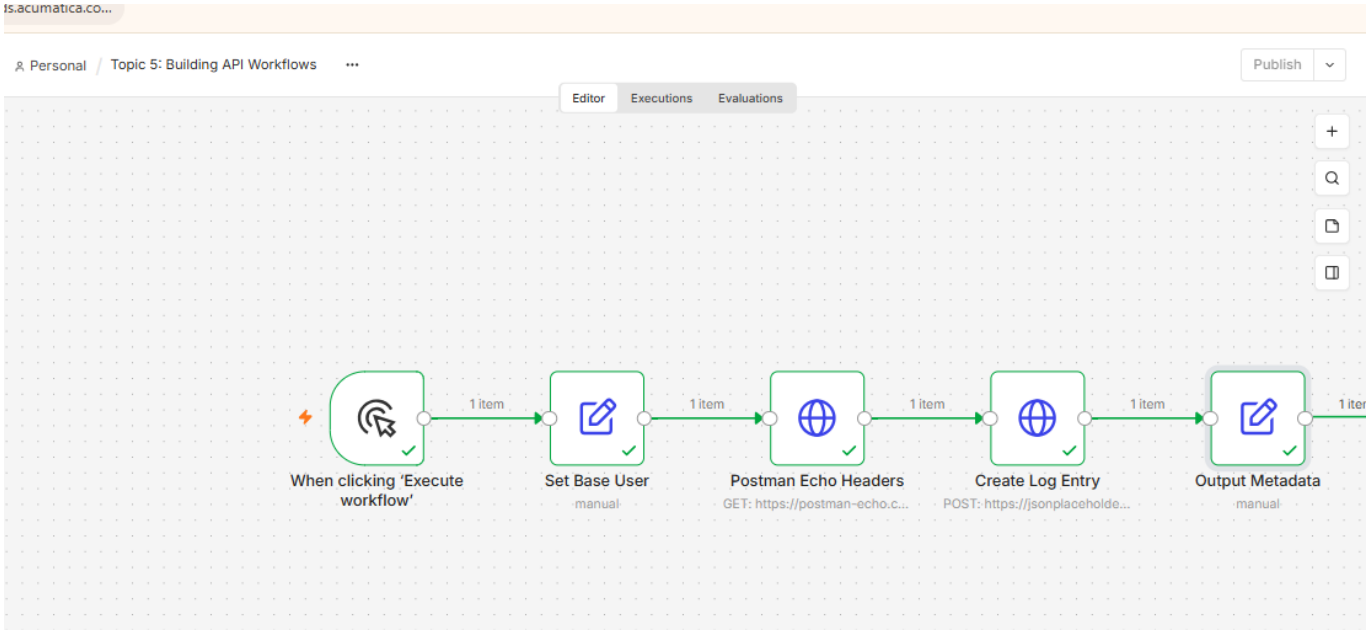
Topic 5 | Building API Workflows

Assessment execution evidence and configuration explanation

Source: five supplied n8n screenshots. Explanations cover visible fields and results. Hidden settings and source lines have not been inferred.

Workflow overview



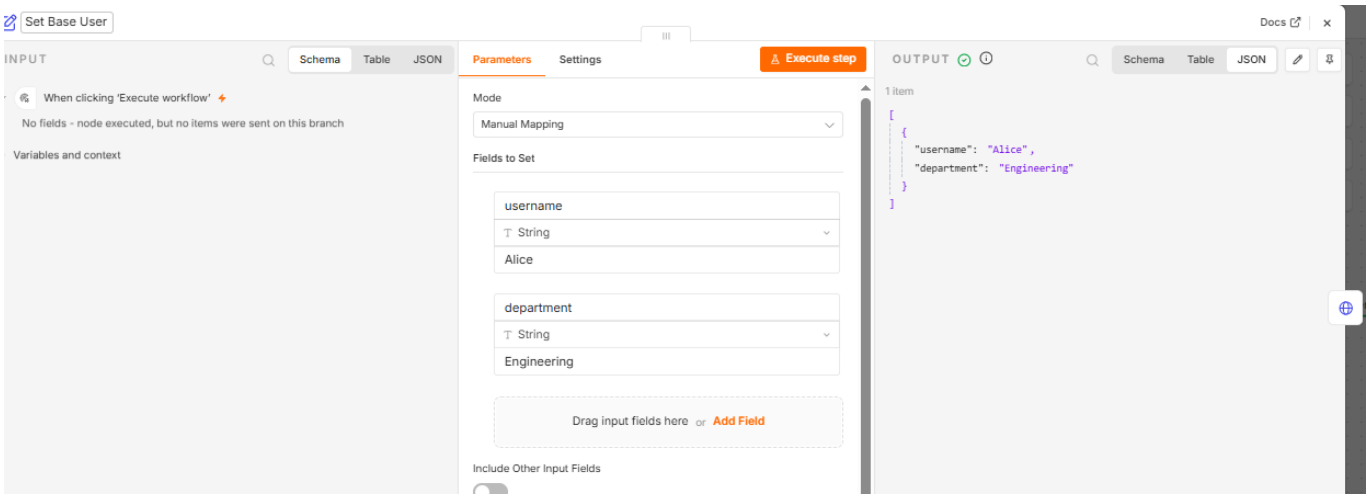
Flow: Manual Trigger -> Set Base User -> Postman Echo Headers -> Create Log Entry -> Output Metadata.

Execution: All five workflow nodes show green success indicators and one item passes between them.

Scope: This is a successful demonstration of API requests, response processing, and execution metadata; it is not a persistent database logging solution.

Screenshot evidence: image(20261030-062820).png

1. Set Base User



Mode: Manual Mapping: Creates named fields directly within the Edit Fields / Set node.

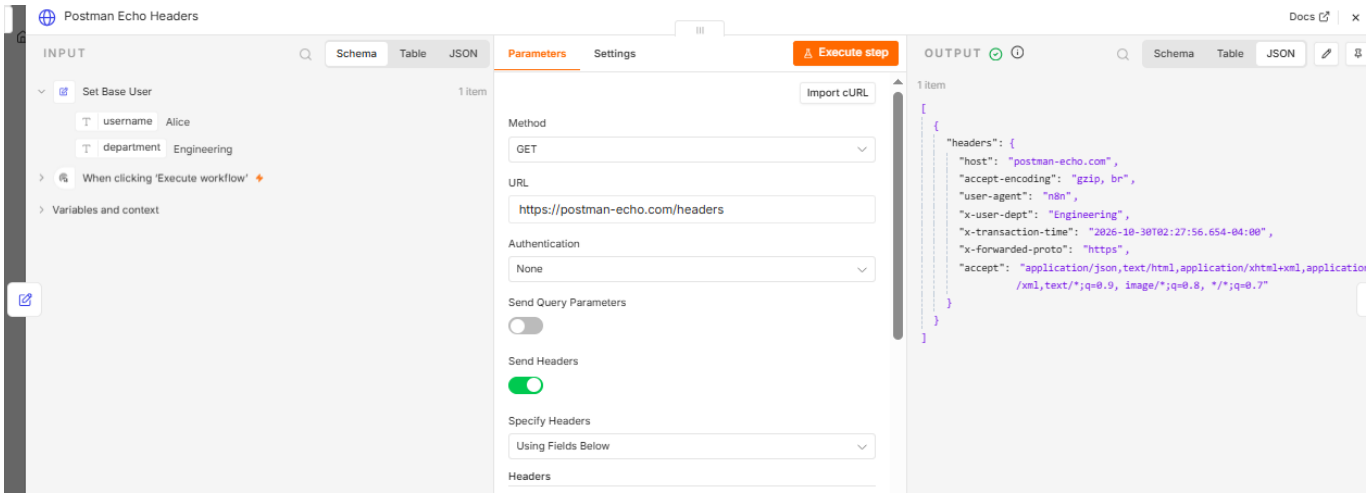
username: Alice: Provides the test user name used later to construct the log title.

department: Engineering: Provides the department value for the custom HTTP header.

Output: one JSON item: The result contains username and department, confirming that mapping worked.

Screenshot evidence: image(20261030-062839).png

2. Postman Echo Headers



Method: GET: Requests data without creating a resource.

URL: https://postman-echo.com/headers: Echoes request headers back in the response for inspection.

Authentication: None: No API authentication is configured for this public test endpoint.

Send Headers: enabled: Allows custom request headers to be sent.

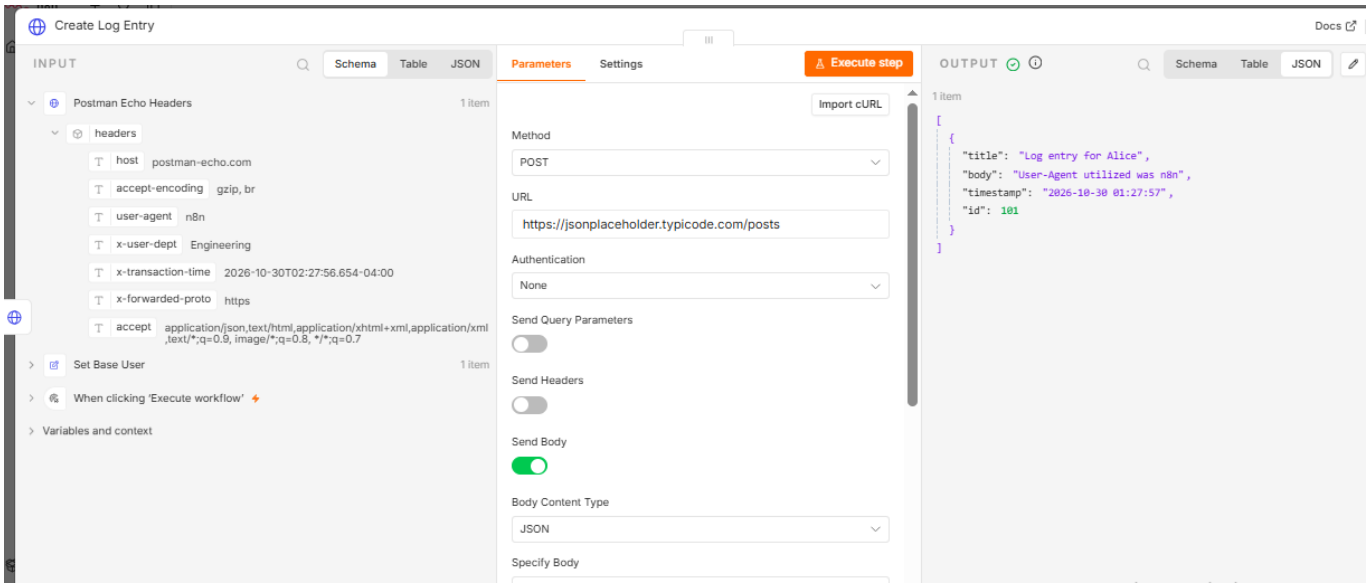
x-user-dept: Engineering: The echoed response confirms that the department header arrived.

user-agent: n8n: The response also reports the client user agent.

Output: headers object: One item was returned with header keys and values.

Screenshot evidence: image(20261030-062855).png

3. Create Log Entry



Method: POST: Submits a JSON body to the example logging endpoint.

URL: https://jsonplaceholder.typicode.com/posts: Uses JSONPlaceholder, a public mock REST API.

Authentication: None: The example endpoint does not require credentials.

Send Body: enabled; content type JSON: The request body is sent as JSON.

title: Log entry for Alice: The response confirms that the request carried the constructed log title.

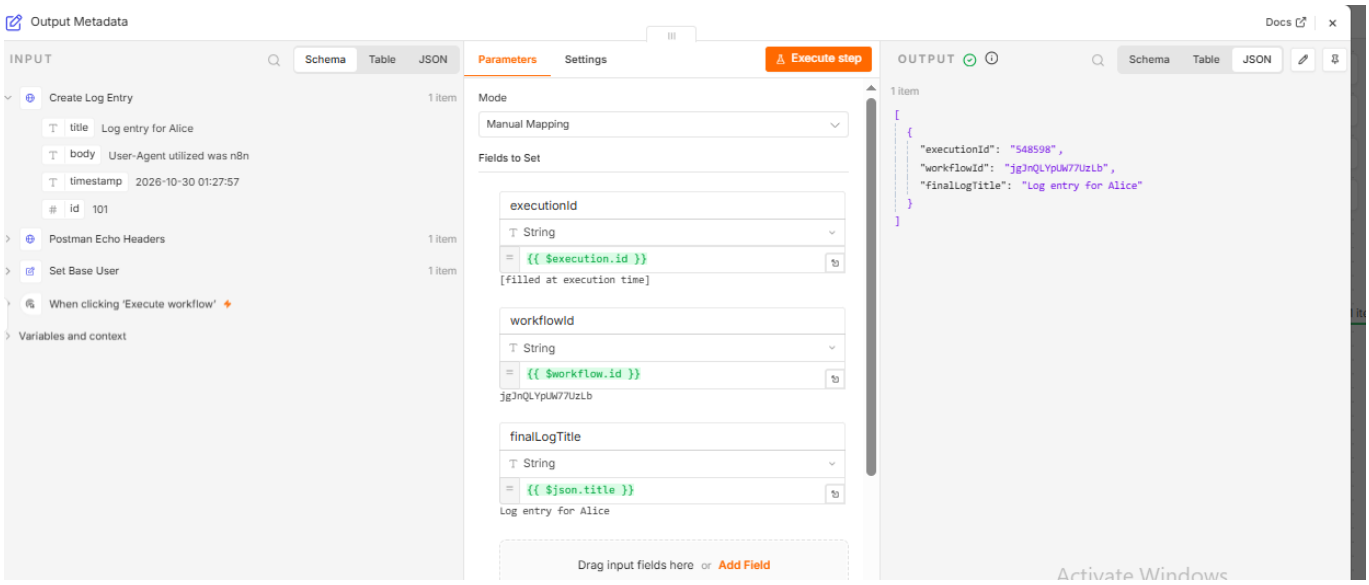
body: User-Agent utilized was n8n: The response records the client-agent detail.

timestamp: The returned JSON includes the submitted timestamp.

id: 101: The mock API returned a generated example resource ID; this does not prove durable storage.

Screenshot evidence: image(20261030-062910).png

4. Output Metadata



Mode: Manual Mapping: Creates a clean final JSON response.

executionId = {{ $execution.id }}: Reads the current n8n execution identifier.

workflowId = {{ $workflow.id }}: Reads the current n8n workflow identifier.

finalLogTitle = {{ $json.title }}: Copies the title from the preceding Create Log Entry response.

Output: The screenshot shows executionId 548598, workflowId jgJnQLyPUW7UzLb, and finalLogTitle Log entry for Alice.

Screenshot evidence: image(20261030-062926).png

Verification and limitations

• All five nodes show successful execution in the workflow overview.

• Set Base User returned Alice and Engineering.

• GET echo response included x-user-dept = Engineering.

• POST mock endpoint returned title = Log entry for Alice and id = 101.

• Final metadata returned executionId = 548598, workflowId = jgJnQLyPUW7UzLb, finalLogTitle = Log entry for Alice.

• A mock POST response does not verify persistence.

• The screenshots do not expose every HTTP header, request-body expression, or hidden configuration line.